

Anik Mollik

ID: IT 22044

Ans to the question no: 01

protected allow access within the same package and in subclass, even if they are in different package.

⇒ Same package Example

```
package mypkg;

public class parent {
    protected int num = 10;
}

class child extends parent {
```

```
    void show() {
```

```
        system.out.println(num);
    }
```

⇒ Different package

```
package parentpkg;
```

```
public class parent {
```

```
    protected int num = 10;
```

```
}
```


package childpkg;

import Parentpkg.Parent;

public class child extends Parents {

void show() {

system.out.println("number"); // Accessible

}}

Ans to the question no: 021

Comparison: Abstract class vs interface

Feature	Abstract class	Interface
Types of member		
Multiple inheritance	Not supported	Supported
Types of member	Can have field, constructor, method	Only static/final & method
Constructor	Yes	No
Access modifier	Supports all (public, protected, etc)	Interface methods are public by default.

When to use:

use an abstract class when:

- You want to share your code among closely related class.
- ② you need to define state (field) or constructor
- ③ you want to provide a base class with some common implementation.
- ④ Only one inheritance is needed.

use an interface when:

- you want to define a contract or capability
- you need to support multiple inheritance
- you are describing "what" a ^{class} should do, not "how"
- you want classes from different hierarchies to share behaviour

Ans to the question no: 03

Encapsulation for data security and Integrity

Encapsulation is a key OOP principle that

- Protected data by making class variable private
- Control class using public getter/setter methods
- Ensuring integrity through validation logic

This way user of the class cannot directly access or modify sensitive data like account number or balance

Bank Account class

Java

```
public class BankAccount {
```

```
    private String accountNumber;
```

```
    private double balance;
```

```
    // set account number with validation
```

```
    public void setAccountNumber(String accNum)
```

```
    { if (accountNumber == null || accountNumber.trim().isEmpty())
```



```
System.out.println("Invalid AccountNumber");  
else { this.accountNumber = accountNumber; } }
```

// set initial balance with validation

```
public void setInitialBalance(double balance)  
{  
    if (balance < 0)  
    {  
        System.out.println("Balance can't be negative");  
    }  
    else { this.balance = balance; }  
}
```

// Getter method

```
public String getAccountNumber()  
{  
    return accountNumber;  
}
```

```
public double getBalance()  
{  
    return balance;  
}
```

// Display account details

```
public void displayAccount()  
{
```

```
    System.out.println("Ac: Num " + accountNumber);
```

```
    System.out.println("Balance $ " + balance);  
}
```


Ans to the question no: 06

DOM (Document object model) Parser

- Load the entire xml into memory as a tree structure
- Allow random access to any part of document
- Ideal for reading and modifying XML content

SAX (Simple API for XML) Parser

- Event driven parser: read xml line by line
- Triggers events (start Element character, end)
- No tree built in memory

Comparison Table

Feature	Dom Parser	SAX Parser
Memory usage	High (load whole xml in memory)	Low
Speed	Slower	Faster for large files
Access	Random access	No Random access
Modifying XML	possible	not possible

Ease of usage | Earlier work | more complex

When to use SAX over DOM

Scenario: You are processing a large log file

In this case:

- DOM use consume too much memory
- SAX read line by line only processed the needed parts
- Faster and more memory efficient

Ans to the question no: 07

React virtual Dom vs Traditional Dom

Traditional Dom (Browser Dom)

- Represent the UI as a tree of HTML element
- Direct manipulation (eg. `document.getElementById()`)
update the UI
- Performance issue: Manipulating the real Dom
- Dom update are expensive
- Re-rendering causes layout recalculation and repainting.

Virtual Dom (Used in React)

- React creates a lightweight in memory copy of actual DOM

- On the state or prop change

1. React create a new virtual Dom

2. It diff the new and previous virtual

3. It calculate the minimum change

4. Then it updates the real dom efficiently

Diffing algorithm

React use for reconciliation algorithm

- Compare old vs new virtual Dom

- Detect what change (text, element attribute)

- Update only change part in the real dom

Component Update

function Greeting() {

return (h1) Hello Alice! {h1}

Virtual DOM vs Real DOM comparison

Feature	Traditional Dom	Virtual dom
Update UI directly	Yes	no
Speed of frequent update	Slower	faster
Whole Dom re-rendered	often	no only what change
Memory usage	Lower	Slower

Ans to the question no: 08

Event Delegation in javascript

Event delegation is a technique where a single event listener is added to a parent element to handle events for current and future child element. It works because of event bubbling - when an event occurs in a child element, it bubbles up through its parent element in the DOM tree.

How it optimized performance

- Fewer event listeners: Instead of adding a listener to every child, you add one to the parent.
- Work with dynamic content: Element added after page load still triggers the parent's listener.
- Reduces memory usage: Avoid redundant listeners on each element.
- Improve performance: Especially important when dealing with large lists.

Click Event on dynamically Added Element

```
<ul id="taskList"> </ul>
```

```
<button id="addTask"> Add task </button>
```

```
</script>
```

```
const taskList = document.getElementById("taskList")
```

```
const addList = document.getElementById("addTask")
```

```
taskList.addEventListener("click", function(event) {  
  if (event.target.tagName === "LI") {
```



```
alert("You clicked: " + event.target.textContent);  
} };
```

Dynamically add task

```
addTask.addEventListener("click", () => {  
  const li = document.createElement("li");  
  li.textContent = "Task" + (taskList.children.length + 1);  
  taskList.appendChild(li);  
});
```

</script>

Ans to the question no: 09

Using Regular Expression for input validation:

Java provide the java.util.regex package which include

Pattern: Compiles a regular Expression

Matcher: Check if a given String matches the pattern

You can use validation input like email, phone number, password.

Validate an email Address
Regex pattern for email Validation

```
String regex = "[A-Za-z0-9+_,-]+@[A-Za-z0-9,-]+\\.?$"
```

start of the string

[A-Za-z0-9+_,-]+ One or more valid character before @

@

The @ symbol separating username and domain

\$

End the string

This regex is simple and covers most common email (like @username1234@)

Java Code Example

```
import java.util.regex.*;  
  
public class EmailValidator {  
    public static void main(String[] args) {  
        String email = "John.doe12@example.com";  
    }  
}
```



```
String regex = "[A-Za-z0-9+_.-]+@
```

```
// compile the pattern
```

```
Pattern pattern = Pattern.compile(regex);
```

```
Matcher matcher = pattern.matcher(email);
```

```
// Check if it matches
```

```
if (matcher.matches()) {
```

```
    System.out.println("Valid email");
```

```
else
```

```
    System.out.println("Invalid email");
```

```
}
```

Workflow

1. `Pattern.compile(regex)` create a compile regex pattern

2. `matcher(email)` creates a matcher for the email

3. `matcher.matches()` return true if the input matches the pattern

Q) + Ans to the question no: 10

Custom annotation in Java are used to add metadata you can attach to class, method, field. They allow you to provide structured information that can be processed at compile time (annotation processor) or Runtime (via reflection).

Why use custom annotation?

- Decouple logic from configuration
- Add metadata that influences behavior
- Used in framework like Spring, Hibernate, JUnit

Define annotation

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)

@Target(ElementType.METHOD)


```
public @interface RunMe {
    String Value() default "Running";
}
```

~~##~~ Progress annotation via reflection

```
import java.lang.reflect.Method;
```

```
public class AnnotationProcessor {
    public static void main(String[] args)
        throws Exception {
```

```
        Class<?> clazz = TestClass.class;
```

```
        Object obj = clazz.getDeclaredConstructor().
            newInstance();
```

```
        for (Method method : clazz.getDeclaredMethods()) {
```

```
            if (method.isAnnotationPresent(RunMe.class)) {
```

```
                RunMe annotation = method.getAnnotation(
                    RunMe.class);
```

```
                System.out.println(" " + annotation.Value());
```

```
                method.invoke(obj);
            }
        }
```


Answer to the question: 11

Singleton

The singleton pattern ensure that only one instance of class is created and provide a global point of access to it?

Problem it solves:

- When only one object is needed to co-ordinate action across a system.
- Prevent multiple instance that can lead to conflicting state.
- Resource contention.
- Unnecessary memory usage.

Basic singleton implementation

public class singleton {

private static singleton instance;

private singleton () {

}


```
public static Singleton getInstance() {
```

```
    if (instance == null) {
```

```
        instance = new Singleton();
```

```
    } return instance
```

```
}}
```

~~# Synchronized Method~~

```
public class Singleton {
```

```
    public static Singleton instance;
```

```
    public Singleton() {}
```

```
    public static synchronized Singleton getInstance()
```

```
    {
```

```
        if (instance == null) {
```

```
            instance = new Singleton();
```

```
        } return instance
```

```
}
```


Ans to the question no: 12

JDBC Java database Connectivity

JDBC is an API that enable java application to interact with relational database. It provide a standard way to

- Connect to the database

- Execute SQL query

- Retrieve and process result

- Handle transaction and error

JDBC workflow for a select query

1. Load JDBC driver

2. Establish connection

3. Create statement

4. Execute SQL Query

5. Process ResultSet

6. Handle Exception

7. Close Resource

Lab-4

Select query with Error Handling

Java import java.sql.*;

```
public class Lab-4SelectExample {  
    public static void main (String [] args) {  
        String url = "jdbc:mysql://localhost:3306/  
mydatabase";
```

```
        String user = "root";
```

```
        String password = "your password";
```

```
        Connection con = null;
```

```
        Statement stmt = null;
```

```
        ResultSet rs = null;
```

```
try { // JDBC driver load
```

```
    Class.forName("com.mysql.cj.jdbc.Driver");
```

```
// 2. Establish connection
```

```
    conn = DriverManager.getConnection(url, user,  
password);
```

```
// create statement
```


Lab 4
stmt = conn.createStatement();

// 9. Execute select query

String query = "SELECT id, name, email
from users";

rs = stmt.executeQuery(query);

// Process Result

while (rs.next()) {

int id = rs.getInt("id");

String name = rs.getString("name");

String email = rs.getString("email");

System.out.println("id + " | " + name + " | " + email

}
catch (ClassNotFoundException e) {
System.out.println("JDBC Driver " + e.getMessage());

catch (SQLException e) {

System.out.println("SQL Error: " + e.getMessage());

} finally { // 6. close resource


```

try { rs != null ) rs.close();
    if (stmt != null) stmt.close();
    if (conn != null) conn.close();
} catch (SQLException e) {
    System.out.println("Error " + e.getMessage());
} } }

```

Ans to the questions: 13

Servlet and JSP in MVC architecture

In java application, Servlets and JSP are often used to implement the MVC pattern.

- Model: Java class (User) handle business logic and data

View: JSP files display HTML output to the user

Controller: Servlet acts as the central coordinator, handling request and response.

MVE flow overview

1. User sends a request (submitting a login form)
2. Servlet (Controller) process the request, uses the model to retrieve or update data
3. Data is added to the request scope and forwarded to a JSP
4. JSP display the result using data

Displaying user Profile

```
public class user {  
    private String name;  
    private String email;  
    public user (String name, String email) {  
        this.name = name;  
        this.email = email;  
    } // Getter  
    public String setName () { return name; }  
    public String getEmail () { return email; }  
}
```



```
# Controller: UserProfileServlet.java
```

```
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
```

```
import java.io.IOException;
```

```
public class UserProfileServlet extends HttpServlet {
```

```
    protected void doGet(HttpServletRequest request,  
        HttpServletResponse response) throws  
        ServletException, IOException {
```

```
        User user = new User("Alice", "alice@example.com");
```

```
        request.setAttribute("user", user);
```

```
        RequestDispatcher dispatcher = request.get
```

```
        RequestDispatcher("profile.jsp");
```

```
        dispatcher.forward(request, response);
```

```
    }
```


Ans

Ans to the question no: 14

Life cycle of a java servlet

The life servlet life cycle of java is managed by the servlet container (like Tomcat) and include the following step

1. Loading and installation

- Servlet container loads the servlet class and create only one instance.

- This typically happens on the first request

2. Initialization

public void init(ServletConfig config)
throws ServletException

- Called once after initialization

- Used to initialize resource like database connection.

- You should not override the constructor - use init() for setup

3. Request Handling (service())

public void service (ServletRequest req, ServletResponse res) throws ServletException, IOException

- called each time a request is received
- It determines the HTTP method (GET, POST) and call the corresponding doGet(), doPost() etc
- This is where the main request program

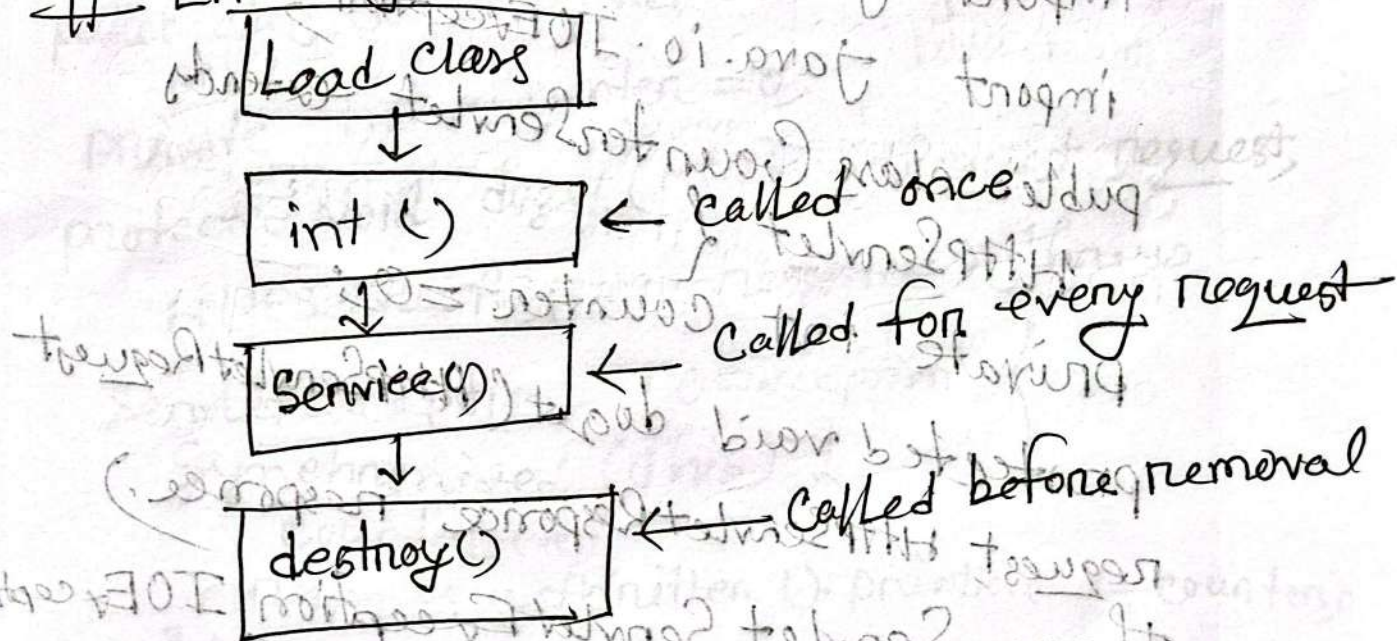
4. Destruction:

public void destroy()

- called once before the servlet is unloaded on the server shuts down.

- Used to release resource

Life Cycle Diagram



Ans to the question no. 15

Shared Resource in multithreaded servlet

In java servlet, the container create one instance of servlet and uses multiple threads to handle multiple concurrent request. If shared resource modified without care, race condition or inconsistent state.

Non Thread Safe Servlet

```
import jakarta.servlet.*;  
import jakarta.servlet.http.*;  
import java.io.IOException;
```

```
public class CounterServlet extends
```

```
    HttpServlet {
```

```
        private int counter = 0;
```

```
        protected void doGet(HttpServletRequest request,  
                                HttpServletResponse response)
```

```
        throws ServletException, IOException {
```



```

{ counter++;
  response.getWriter().println("visitor
    number + counter);
}

```

- ~~What could go wrong~~
- two threads may read the same value of counter before their increment it
 - This leads to missed increment giving incorrect visitor counts
 - It's a classic race condition

~~Solution: Use Synchronized block~~

```

public class CounterServlet extends HttpServlet {
    private int counter = 0;
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws
        ServletException, IOException {
        synchronized (this) {
            counter++;
            response.getWriter().println(" + counter);
        }
    }
}

```


Ans to the question no: 16

The MVC (Middle view controller) pattern is a separate concern in java web application by dividing it into three component.

- Model — Manage data business logic
- View — Handle presentation
- Controller — Process user input

Advantages in student Registration form

1. Maintainability — Change in UI don't affect business logic, and vice versa
2. Scalability — Component can be updated independently.
3. Reusability — Model can be reused across different controllers or application.

Lab-5

Ans to the question no 17

In java EE application, a servlet controller manages the flow between the model and the view by

1. Processing Request (eg from submission)

2. Interacting with the model (e.g. fetching data from database)

3. Forwarding data to the flow view (JSP)

using `request.setAttribute()`

4. Rendering the response via `RequestDispatcher.forward()`.

Displaying student details

1. Servlet (controller)

@WebServlet("/studentdetails")

```
public class StudentServlet extends HttpServlet {
```

```
protected void doGet (HttpServletRequest request,
```

```
HttpServletResponse response)
```

```
throws ServletException, IOException) {
```


// fetch data from model

Student student = StudentDAO.getStudentByID

// pass data to the view

request.setAttribute("student", student)

// forward to JSP

RequestDispatcher dispatcher

=> request.getRequestDispatcher("stud

dispatcher.forward(request, response)

}

JSP view

<html>

<head>

<h1> Student Information </h1>

<p> ID: {student.id} </p>

<p> Name is {student.Name} </p>

<p> Email: {student.email} </p>

</body> </html>

Ans to the question no: 18

Compare of session Tracking Method in Servlet

Method	How it works	Advantage
Cookies	Stores session data as a key value pair in the client's browser	Simple to implement - No server side storage needed
URL Rewriting	Append session ID to URL	Works even if cookie are disabled No server storage overhead
HTTP Session	Store session data on the server identified by session cookie	- Secure - No size limit - Built in session management

Ans to the question-19

In Java EE application, a servlet controller acts as the intermediary between the Model & View. It processes request, interacts with the model and forwarding data to a JSP for rendering.

Forwarding Data from Servlet to JSP

1. Servlet

@WebServlet("/user")

public class UserServlet extends HttpServlet {
protected void doGet (HttpServletRequest request,
HttpServletResponse response)

throws ServletException, IOException {

// Model interaction

String userName = "John Doe";

request.setAttribute("name", userName);

request.getRequestDispatcher("/userView.jsp").

forward(request, response);

}}

2. JSP (view) → userview.jsp

```
<html>
<head>
  <title>User Page </title>
</head>
```

```
<body>
  <h1>Welcome, $name! </h1>
```

```
</body>
</html>
```

Ans to the question 20:

Key Component and their Roles

1. Controller

- Mark a class as a request handler
- Contain method that process HTTP Request
- Separate business logic

2. @RequestMapping

- Maps HTTP Request
- Define URL paths
- Can be narrowed down using

3. Model Object

- Carrier data between controller and view
- Act as a container attribute
- Separate data from rendering logic

Login from submission

1. Controller process the request

@Controller

```
public class LogicController {
```

```
@GetMapping ("/login")
```

```
public String Showloginform (Model model)
```

```
model.addAttribute ("loginRequest", new loginRequest)
```

```
return "login"
```

```
}
```

```
@PostMapping ("/login")
```

```
public String Handlelogin (
```

```
@ModelAttribute ("loginRequest")
```

```
loginRequest loginRequest,
```

```
Model model.)
```

```
{ if (isValidUser (loginRequest, loginRequest))
```

```
{ loginRequest.setPassword ("")
```

```
model.addAttribute ("user", loginRequest.getUsername())
```




return "login";

private boolean isValidUser() {

String username, String password;

return username.equals("admin");

password.equals("1234");

}

<form action="login" method="post">

<input type="text" name="username" placeholder="Username" />

<input type="password" name="password" placeholder="Password" />

<button type="submit"> Login </button>

</form>

Ans to the question no-21

In spring MVC, the dispatcher servlet acts as the front controller - a core component that manages the entire processing workflow. It is responsible for receiving all incoming web requests and directing the flow between different components like controllers, handler mapping.

• Role of dispatcher servlet in request

Workflow:

1) client request

A user sends a request (e.g. /home) to the server.

This request is intercepted by the dispatcher servlet, which is configured in web.xml (or auto-configured in

Boot.)

2) Handler mapping

The dispatcher servlet uses Handler mapping to find the appropriate controller that matches the request URL.

3) Controller Invocation:

The controller method is called.

- It processes the request and returns

either a view name or a Model And view object (containing view + model data).

4) View resolution:

- The returned view name (like "home") is passed to the View Resolver.

- The View Resolver maps the logical view name to an actual file (e.g., /WEB-INF/views/home.jsp).

5) View rendering

- The view is rendered with model data.

The final HTML output is returned to the client.

Example:

1) User request: GET/greet

2) Dispatcher Servlet receives it.

3) Handler Mapping → maps to GreetingController.

greetUser()

4) Method returns Model And View ("greeting", model)

5) view is rendered → sent back to browser.

Answer no - 22

PreparedStatement vs Statement in

Advantages of PreparedStatement:

- Performance: SQL is precompiled, database caches the query plan, so repeated execution is faster.
- Security: Prevents SQL injection, parameters are bound safely and not concatenated.

Example: Insert record with PreparedStatement.

```
Java
import java.sql.*;

public class InsertExample {
    public static void main(String[]) {
        String url = "jdbc:mysql://localhost:3306/";
        String user = "root";
        String password = "1234";

        try {
            Connection conn = DriverManager
                .getConnection(url, user, password);
            String sql = "INSERT INTO student (age) VALUES (?, ?)";
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```



```

PreparedStatement pstmt = conn. prepared
Statement (sql);
pstmt. setString (1, "Alice");
pstmt. setInt (2, 20);
int rows = pstmt. executeUpdate ();
System. out. println (rows + " row(s) inserted.");
} catch (SQLException e) {
    e. printStackTrace ();
}
}

```

Answer NO-23

ResultSet in JDBC

Definition

ResultSet is a table of data returned by executing a query. It provides methods to methods to navigate rows and read column values.

- next() → Moves the cursor to the next row
- Returns false if no more rows.

• `getString(columnLabel)` → Retrieves a column value as a string.

• `getInt(columnLabel)` → Retrieves a column as an integer.

Example: Retrieve student data

java

```
import java.sql.*;  
public class ResultSetExample {  
    public static void main (String[] args)
```

```
    {  
        String url = "jdbc:mysql://localhost:3306/school";
```

```
        String user = "root";
```

```
        String password = "1234";
```

```
        try (Connection conn = DriverManager.
```

```
            getConnection (url, user, password))
```

```
        {  
            Statement stmt = conn.createStatement();
```

```
            ResultSet rs = stmt.executeQuery("SELECT  
            name, age FROM students");
```

```
            while (rs.next()) {
```

```
                int id = rs.getInt("id");
```

```
                String name = rs.getString("name");
```



```

int age = rs.getInt("age");
System.out.println(id + " | " + name + " | " + age);
}
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

Answer No-24

JPA Mapping
 JPA maps Java classes to relational tables
 using annotations.

Example

java

import jakarta.persistence.*;

@Entity

public class Student {

@Id

@GeneratedValue(strategy = GenerationType.

IDENTITY)

private Long id;


```

private String name;
private int age;
// getters and setters
}

```

How it works

- @Entity → Marks this class as a table
- @ID → specifies primary key.
- @GeneratedValue → Auto-generate primary key values.

- Advantages over JDBC
- No need for SQL queries → use object operations.
 - Database-agnostic
 - Built-in transaction & relationship management.

@Entity
 public class Student {
 @ID
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

Answer NO-25

EntityManager operations

- `persist(entity)` → Adds a new entity (INSERT).
use when adding a new record.
- `Merge(entity)` → Updates existing or saves if detached.
use when modifying an existing record.
- `remove(entity)` → Deletes entity.
use when deleting a record.

Example Scenario:

java

```
em.getTransaction().begin();
```

```
Student s = new Student();
```

```
s.setName("Alice");
```

```
s.setAge(20);
```

```
em.persist(s); // CREATE
```

```
s.setAge(21);
```

```
em.merge(s); // UPDATE
```

```
em.remove(s); // DELETE
```

```
em.getTransaction().commit();
```


Answer NO-26

Spring Boot CRUD with MySQL

Entity

java

@Entity

```
public class Student {
```

@Id

@GeneratedValue (Strategy = GenerationType
IDENTITY)

private long id;

private String name;

private int age;

}

→ Repository

java

import org.springframework.data.jpa.

repository.JpaRepository;

public interface StudentRepository extends

JpaRepository<Student, Long> {

→ Service/Controller

@RestController

@RequestMapping("/students")

public class StudentController {

@Autowired


```
private StudentRepository repo;
```

```
// CREATE
```

```
@PostMapping
```

```
public Student create(@RequestBody Student student){  
    return repo.findAll();  
}
```

```
}
```

```
// UPDATE
```

```
@PutMapping("/{id}")
```

```
public Student update(@PathVariable Long id,  
@RequestBody Student student){  
    student.setId(id);  
    return repo.save(student);  
}
```

```
}
```

```
// DELETE
```

```
@DeleteMapping("/{id}")
```

```
public void delete(@PathVariable Long id){  
    repo.deleteById(id);  
}
```

```
}
```


Answer No-27

Spring Boot RESTful Services

Spring Boot simplifies by:

- Auto-configuring JSON handling (via, Jackson)
- Using annotations to map endpoints.

Example

Java

@RestController

@RequestMapping("/api")

public class HelloController {

@GetMapping("/greet")

public String greet() {

return "Hello, world!";

}

@PostMapping("/student")

public Student createStudent(@RequestBody
Student student) {

return student;

}

}

Answer No-28

@ Rest Controller vs @ Controller

- @ Rest Controller → Combines @ controller & @ Response Body. Returns JSON/XML directly
- @ Controller → Returns view (JSP/Thymeleaf).

Library API End points

- GET /books → List all books
- GET /books / {id} → Get one book
- POST /books → Add new book
- PUT /books / {id} → Update book
- DELETE /books / {id} → Delete book

Answer No-29

Maven in Spring Boot

- Dependency management via pom.xml.
- Build lifecycle: phases → validate → compile → test → package → install → deploy.

Example pom.xml

xml

<project>

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>student-app2</artifactId>

<version>1.0.0</version>

<parent>

<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>

<version>3.2.2</version>

</parent>

<dependencies>

<!-- web starter -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>

</dependency>

<!-- JPA starter -->

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<!-- mysql driver -->

<dependency>

<groupId>mysql</groupId>

<artifactId>mysql-connector-j</artifactId>

</dependency>

</dependencies>

</project>

Answer No-30

- Maven: XML-based, convention over configuration, stable.
- Gradle: Groovy/Kotlin DSL, faster builds, more flexible.

Gradle Example

groovy

plugins {

id 'org.springframework.boot' version '3.2.2'

id 'io.spring.dependency-management' version

'1.1.0'

id 'java'

}

group = 'com.example'

version = '1.0.0'

sourceCompatibility = '17'

repositories {

mavenCentral()

}

dependencies {

implementation 'org.springframework.boot:spring

-boot-starter-web'

implementation 'org.springframework.boot:spring

-boot-starter-data-jpa'

run time only 'mysql:mysql-connector-j'
test Implementation 'org.springframework.boot-spring-boot-starter-test'

Answer No-31

Multi-Module Spring Boot project

Structure

css

student-app/

├ api / (REST controllers)

├ service / (business logic)

├ persistence / JPA entities, repositories

└ pom.xml

parent pom.xml

xml

<modules>

<module> api </module>

<module> service </module>

<module> persistence </module>

</modules>

Each submodule has its own pom.xml

Answer No- 32

Project Demonstration

You'd show:

- Code: Entities, Repository, Controllers (like Lab 7 example).
- GUI: Could use Thymeleaf, React or Angular
- Example:
 - Webpage form to add student.
 - Table to display all students
 - Edit / Delete buttons.